

VYŠŠÍ ODBORNÁ ŠKOLA A STŘEDNÍ PRŮMYSLOVÁ ŠKOLA
ELEKTROTECHNICKÁ FRANTIŠKA KŘÍŽÍKA, NA PŘÍKOPĚ 16,
PRAHA 1

ZÁVĚREČNÁ PRÁCE
Z PŘEDMĚTU:

Robotika

Pololetní projekt

Jméno a příjmení: Shusharin Vladyslav a Mergl Vladimír

Studijní obor: Informační a komunikační technologie

Obsah

1	Úvod	2
2	Teoretická část.....	3
2.1	Návrh konstrukce.....	3
2.2	Vývojový diagram	4
3	Praktická část.....	9
3.1	Praktická konstrukce.....	9
3.2	Praktická tvorba programu	11
4	Manuál.....	22
5	Závěr.....	24
6	Seznam použitých informačních zdrojů	25
7	Seznam příloh	26
8	Seznam obrázků.....	26

1 Úvod

DnD robot neboli robot, s kterým bude uživatel bojovat, je humanoidní robot z Lega a elektrických součástek od LEGO Mindstorms. Tento robot bude mít dvě proměnné, životy uživatele a životy robota. Robot bude náhodně útočit a uživatel bude vybírat svojí akci pomocí barevných kostek, které robot bude moci detekovat barevným senzorem. Podle vybrané akce uživatele program bude vykonávat různé úkony. Když hráč například zaútočí, tak se uberou robotovi životy. Robot také například vykoná určitou animaci při jeho akci. Souboj bude probíhat jako typická RPG hra, tahově.

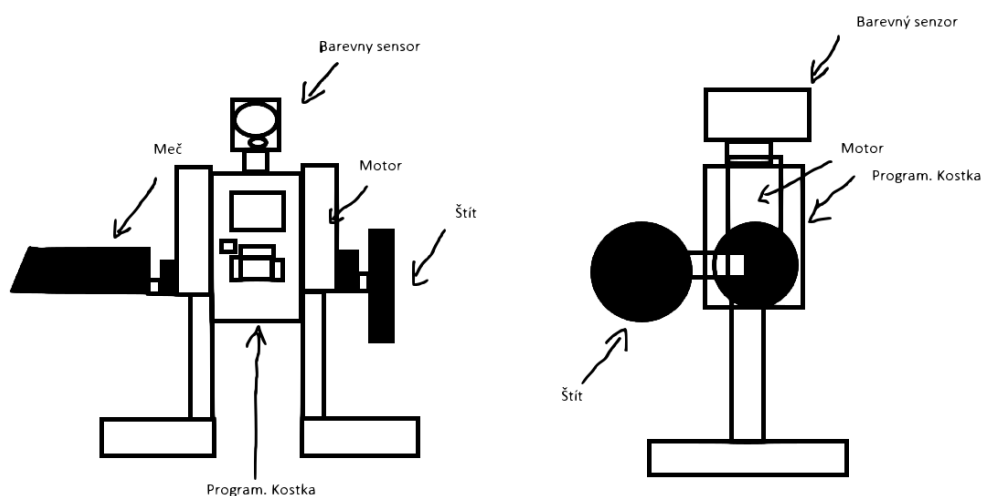
Byl vybrán tento nápad, protože je více zaměřený na samotný program a interakci s hráčem. Podle rozhodnutí by bylo víc zábavné, když robot vyžaduje, aby uživatel s ním interagoval, a musel se i zamyslet, jak vyhrát proti robotovi. Robot, který reaguje na výběr akcí hráče by měl dávat pocit, že robot doopravdy se brání proti uživateli.

Inspirace pochází ze stolní hry Dungeons and Dragons a všeobecných RPG her, ve kterých probíhá boj tahově. Bylo rozhodnuto, že tento styl souboje je nejlepší pro vypracování z pohledu stavení samotného robota a nenáročný pro uživatele, který si bude s tímto robotem hrát.

2 Teoretická část

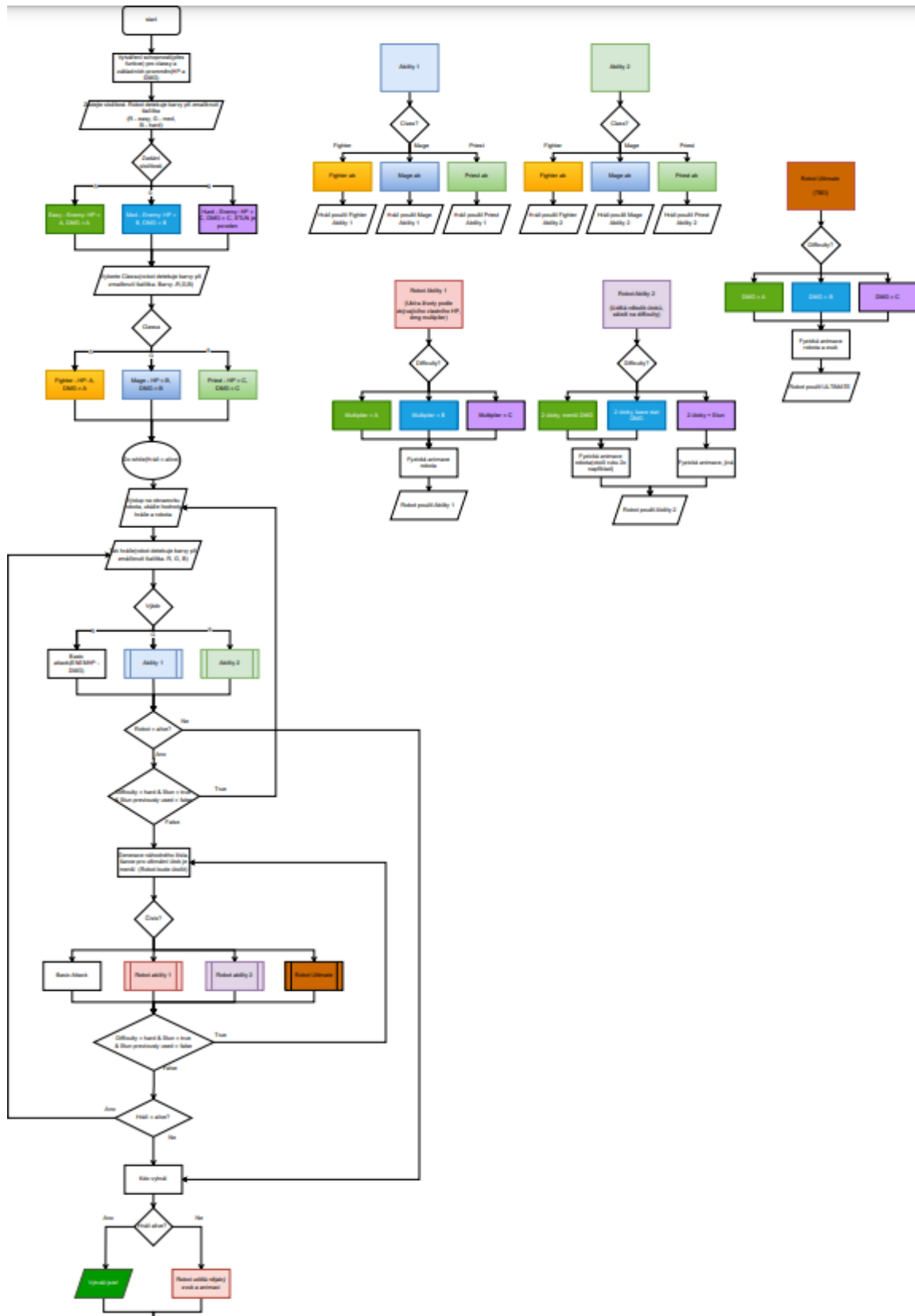
2.1 Návrh konstrukce

Robot je sestaven na dvou čtvercových nohách s EV3 programovou kostkou veprostřed. Po stranách má motory, na kterých jsou připevněny modely štítu a meče z různých součástek, které máchají při útoku na hráče. Na kostce zezadu je připevněný barevný senzor, který spolu fungují jako ovládání ze strany hráče. EV3 kostka je upevněna na nohách s čtvercovými chodidly, aby nemohla spadnout, poněvadž motory nejsou až tak lehké a při jejich pohybu určitě může nastat nestabilita.

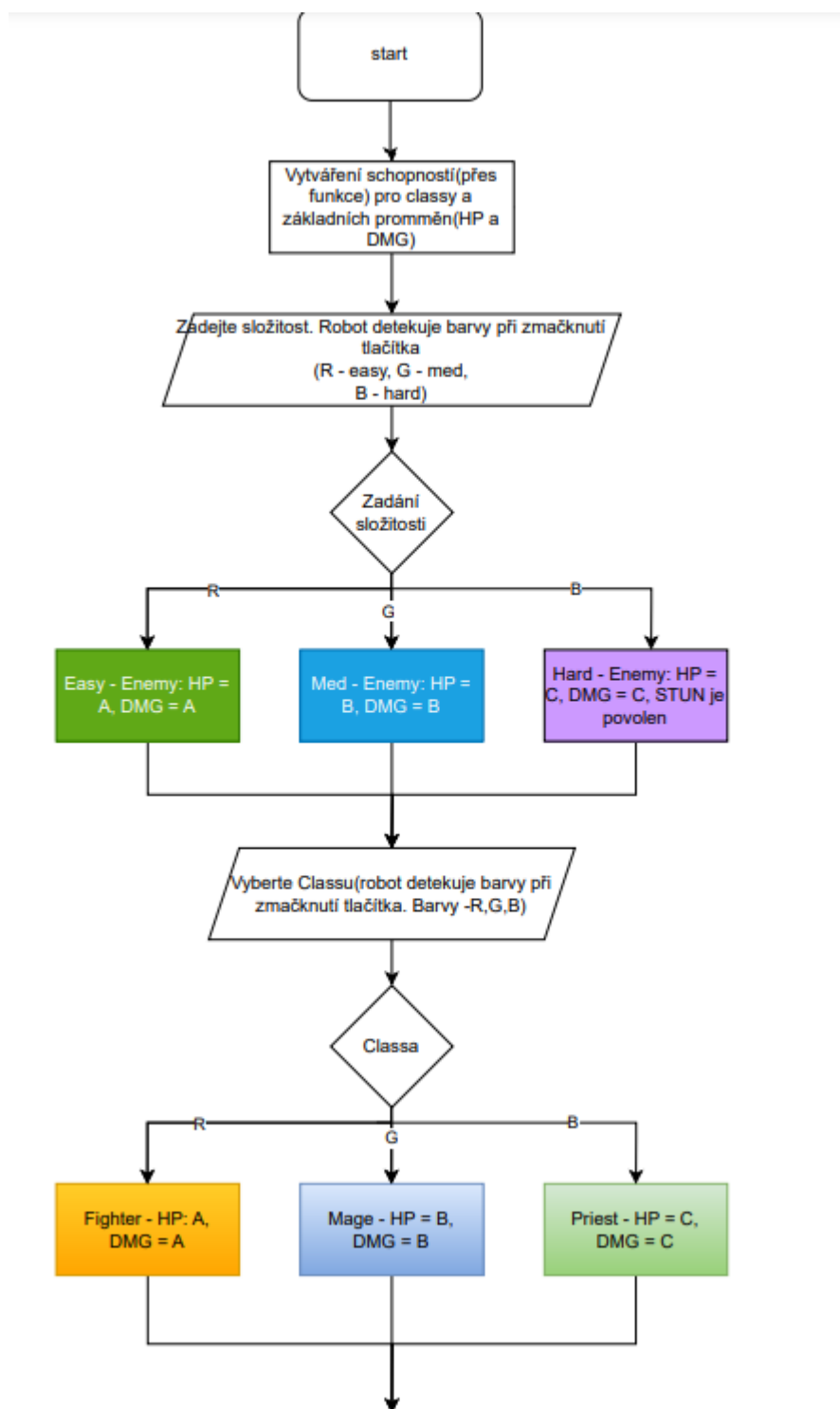


Obrázek 1 - Nakreslený návrh konstrukce

2.2 Vývojový diagram

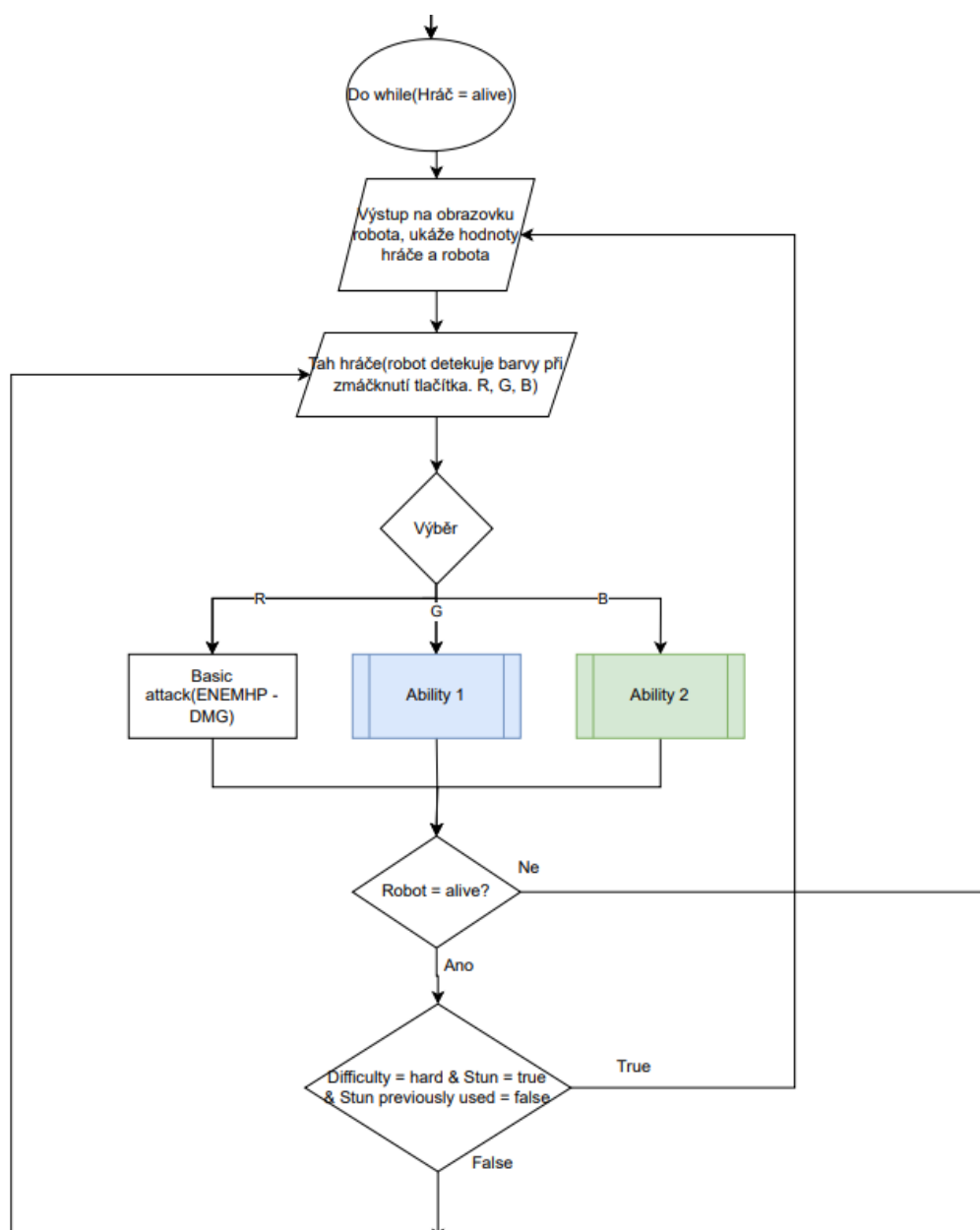


Obrázek 2 - Celý vývojový diagram



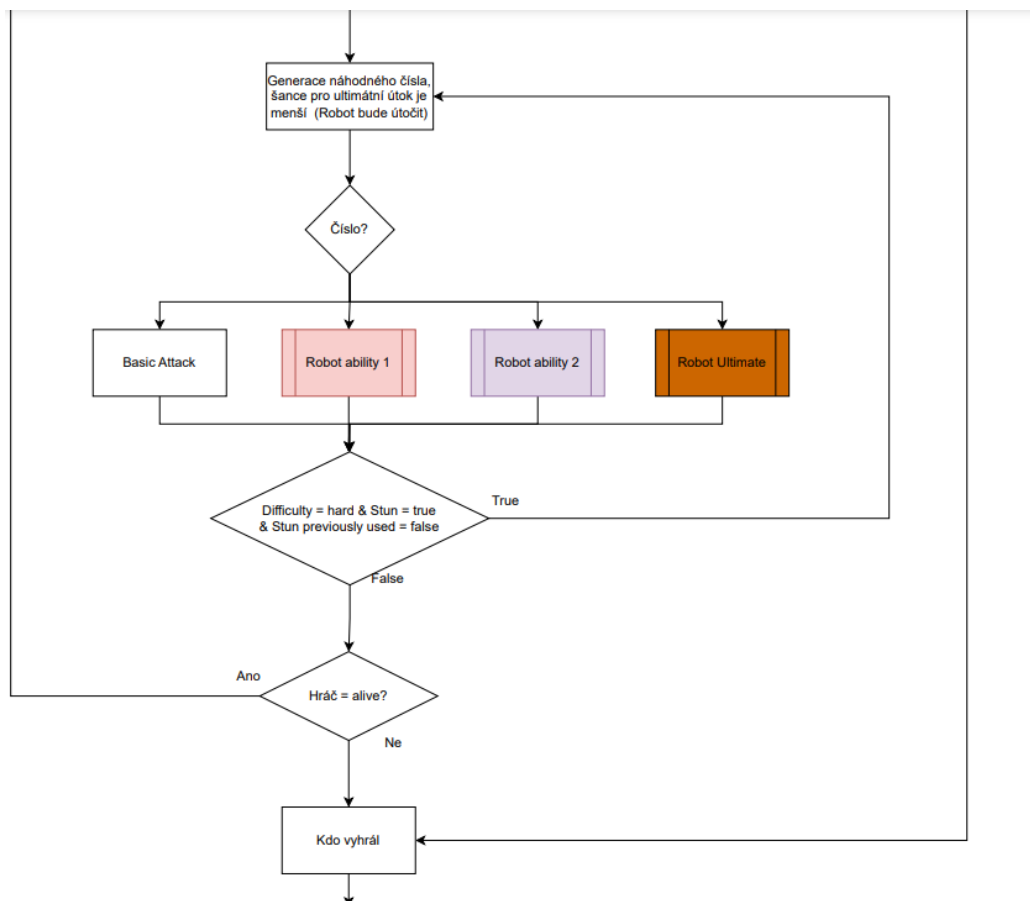
Obrázek 3 - Začátek diagramu

Program, před začátkem hlavního úkolu tohoto robota, se zeptá hráče pro složitost a klasu (Fighter, Mage, Priest). Při vstupu uživatel musí stisknout tlačítko, toto je pro barevný senzor, aby program nepokračoval bez vstupu uživatele. Je to tak vymyšleno, protože pořad zapnutý barevný senzor by detekoval náhodné barvy a pokračoval v programu dál, je možné, že celý program by nevhodně skončil nebo se pokazil. Toto je zajištěno u každého vstupu od uživatele.



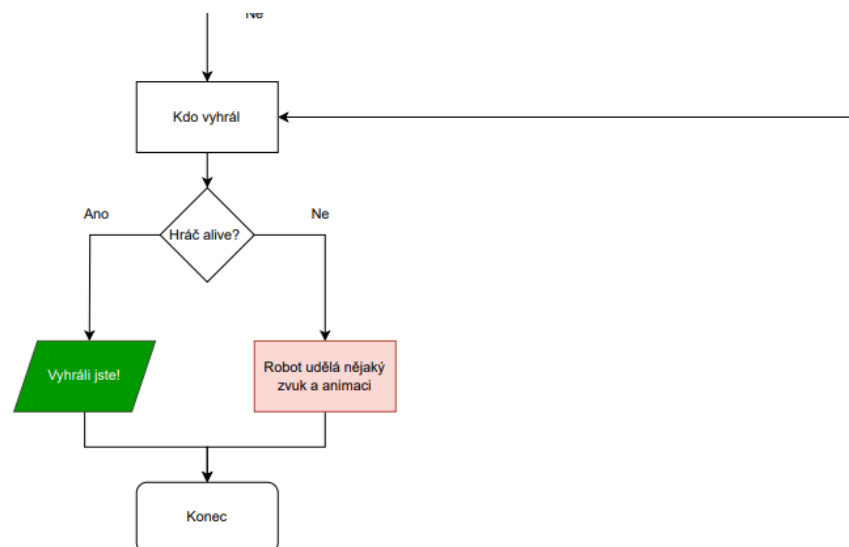
Obrázek 4 - Tah hráče v diagramu

Potom program začne s hlavním úkolem. Boj probíhá tahově, takže po každém tahu hráče a tahu robota se program v této části opakuje. Nejprve se budou vypisovat hodnoty na malou obrazovku robota ukazující životy hráče a robota a výběr pro uživatele, kde jsou vypsány barvy napřed, aby hráč věděl, jakou barvu má ukázat. V tomto úseku jsou také funkce pro „stun“ mechaniku, která funguje jenom při složitosti „Hard“. Když je použita schopnost (hráče nebo robota), která tuto mechaniku má, tak ten, kdo jí použil, dostane další tah. Pokud se schopnost se „stun“ mechanikou použije zase, tak „stun“ nebude fungovat, a další tah bude pro další osobu na řadě.



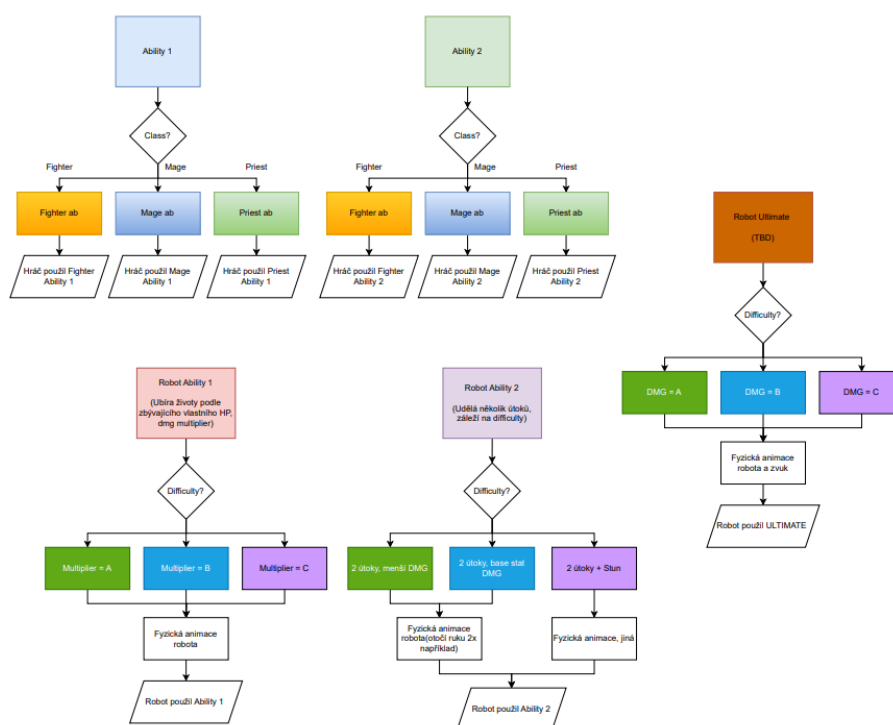
Obrázek 5 - Tah robota v diagramu

Robotův tah vypadá tak, že program vygeneruje náhodné číslo a robot použije korespondující schopnost. Robot také má menší šanci použít ultimátní schopnost. Schopnosti se také trochu mění podle vybrané složitosti. Například robot bude dělat větší poškození, nebo více útoků a „stun“. V programu jsou také vyrobeny funkce pro volání a používání daných schopností.



Obrázek 6 - Konec diagramu

Tento cyklus běží, dokud někdo neumře. Pokud hráč zemře, tak program to vypíše a robot udělá určitou animaci a nějaký zvuk a ukončí se program. Pokud zemře robot, tak vyhraje hráč a program se zase ukončí.



Obrázek 7 - Nadefinované funkce v diagramu

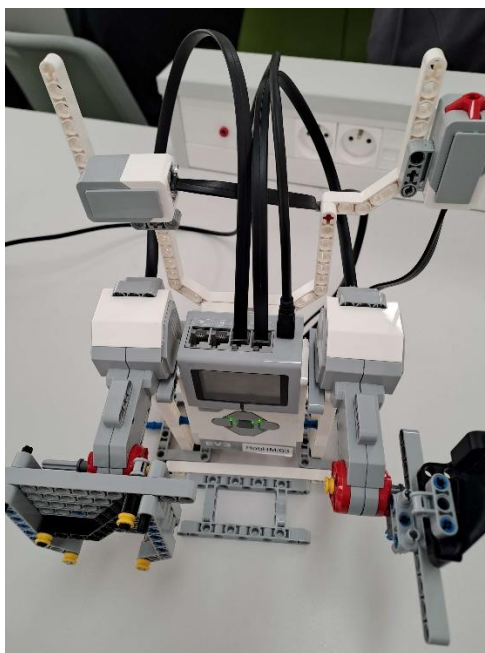
Zde jsou funkce, které v programu se využívají pro výběr hráče podle vybrané postavy a funkce pro útoky robota. Použití funkcí usnadňuje čitelnost a práci v samotném programu.

3 Praktická část

3.1 Praktická konstrukce

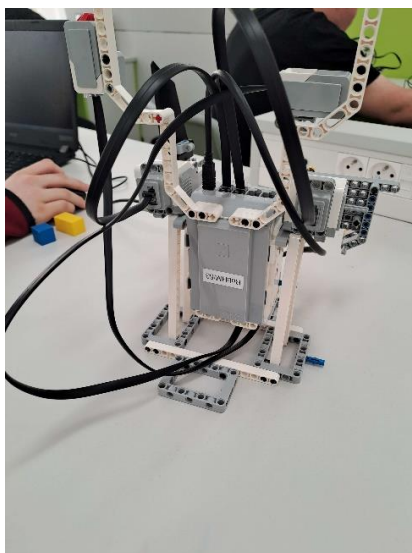
Konstrukce robota je poměrně jednoduchá a nenáročná. Robot obsahuje meč a štít připojený na dva motory ze stran, ty hýbe při určitých úkonech v souboji s uživatelem. Je postaven na čtvercované ploše, aby nespadl. Na sebe má připojené „rohý“, na kterých jsou dva senzory. Senzor barev a tlačítkový senzor pro stisk. Tlačítkový senzor a senzor barev fungují spolu jako ovládání pro uživatele, pro interakci s robotem.

Samozřejmě při konstrukci docházelo k některým složitostem, například dělení štítu robota. Štít je poskládán tak, aby robot mohl ho zvedat, tedy točit motorem k sobě. Problém byl v tom, že štít se někdy nepodařil správně sestavit, aby nebyl příliš velký. Ale největší problém byl spíš ten, že štít byl připojen tak, že se mohl točit náhodně do stran. Toto bylo vyřešeno pomocí lepšího připojení na motor pomocí trochu víc součástek.



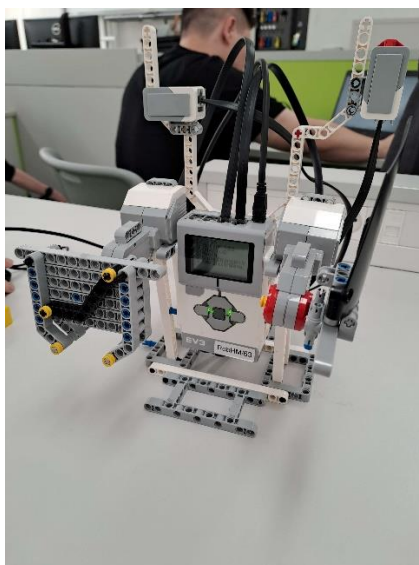
Obrázek 8 - Kabely robota seshora

Zde jsou připojené kabely motorů na robota. Byly vybrány kratší kabely, protože motory jsou nejbližší k jejich portům. Motory byly docela lehce připojeny na strany kostky.



Obrázek 9 - Kabely robota zezadu

Zezadu jsou vidět delší kabely putující seshora dolů. Kabely senzoru jsou tedy připojené do dolních portů, proto byly vybrány delší kabely, jinak by nešlo připojit senzory ke správným portům.



Obrázek 10 - Finální konstrukce

Zepředu jsou vidět všechny hlavní komponenty robota. Meč a štít na motorech a senzory pro interakci s robotem. Robot na displeji ukazuje text a hodnoty při tahovém souboji s uživatelem. Toto je konečná verze konstrukce robota.

3.2 Praktická tvorba programu

```
#!/usr/bin/env pybricks-micropython
from pybricks.hubs import EV3Brick
from pybricks.ev3devices import (Motor, TouchSensor, ColorSensor,
                                  InfraredSensor, UltrasonicSensor, GyroSensor)
from pybricks.parameters import Port, Stop, Direction, Button, Color
from pybricks.tools import wait, StopWatch, DataLog
from pybricks.robotics import DriveBase
from pybricks.media.ev3dev import SoundFile, ImageFile
import time
import random

#This program requires LEGO EV3 MicroPython v2.0 or higher.
# Click "Open user guide" on the EV3 extension tab for more information.

# Create your objects here.
ev3 = EV3Brick()
```

Obrázek 11 - Knihovny programu

```
ev3 = EV3Brick()

# Write your program here.

color_sensor = ColorSensor(Port.S2)
button = TouchSensor(Port.S1)
motorsword = Motor(Port.D)
motorshield = Motor(Port.C)

Difficulty = "None"
Class = "None"

EnemyHP = 0
EnemyDMG = 0
EnemyHPFull = 0
EnemyTurn = False
ultimate = 0

PlayerHP = 0
PlayerDMG = 0
PlayerHPFull = 0
PlayerMANA = 0
PlayerMANAFull = 0
Manacost1 = 0
Manacost2 = 0

Stun = False
StunUsedEnemy = False
StunUsedPlayer = False

Proceed = False
Basic = "N"
Ab1 = "N"
Ab2 = "N"
```

Obrázek 12 - Nadefinované hodnoty

Na začátku kódů jsou uvedeny použité knihovny, které jsou už předepsané EV3 microPython rozšířením, ale „import time“ a „import random“ jsou manuálně napsané. Tyto dvě knihovny jsou dále v kódu využity pro určité funkce, které jsou potřebné pro tento program.

Dále jsou deklarované proměnné a zařízení, která robot využívá, korespondující k jejich fyzickému připojení. Proměnné typu „integer“ slouží v kódu hlavně jako čísla, která se mění při hlavních úkolech robota, tedy souboje (životy a mana). Některé další proměnné jsou typu „string“, které slouží k jmenování postav a schopností. Také jsou tu proměnné, které slouží k využití jako kondice v cyklech a pro ověřování určitých akcí, které uživatel bude konat, většinou jsou to proměnné typu „boolean“ (např. Enemyturn, Stun), ale také i „integer“ proměnné (např. PlayerHPFull, PlayerMANAFull).

```
def ability1(PlayerHP, PlayerDMG, PlayerFullHP, EnemyHP, EnemyFull, PlayerMANA, Stun):

    if Class == "Fighter":
        ev3.screen.draw_text(1,50, "Used Brute Strength")
        ev3.speaker.say("Ow")
        time.sleep(1)
        ev3.screen.clear()
        ev3.screen.draw_text(1,50, "Dealt" + str((PlayerDMG+PlayerHP*0.15)) + " DMG")
        EnemyHP = EnemyHP - PlayerDMG + PlayerHP*0.15
        PlayerMANA = PlayerMANA - 20

    if Class == "Mage":
        if EnemyHP >= EnemyFull * 0.2:
            ev3.screen.draw_text(1,50, "Shot 1 Mana Orb")
            ev3.speaker.say("Ow")
            time.sleep(1)
            ev3.screen.clear()
            ev3.screen.draw_text(1,50, "Dealt" + str((PlayerDMG * 1)) + " DMG")
            EnemyHP = EnemyHP - PlayerDMG * 1
            PlayerMANA = PlayerMANA - 30

        elif EnemyHP >= EnemyFull * 0.5:
            ev3.screen.draw_text(1,50, "Shot 2 Mana Orbs")
            ev3.speaker.say("Ow")
            time.sleep(1)
            ev3.screen.clear()
            ev3.screen.draw_text(1,50, "Dealt" + str((PlayerDMG * 2)) + " DMG")
            EnemyHP = EnemyHP - PlayerDMG * 2
            PlayerMANA = PlayerMANA - 30

        elif EnemyHP >= EnemyFull * 0.8:
            ev3.screen.draw_text(1,50, "Shot 3 Mana Orbs")
            ev3.speaker.say("Ow")
            time.sleep(1)
            ev3.screen.clear()
            ev3.screen.draw_text(1,50, "Dealt" + str((PlayerDMG * 3)) + " DMG")
            EnemyHP = EnemyHP - PlayerDMG * 3
```

Obrázek 13 - Funkce postav 1

```
        elif EnemyHP >= EnemyFull * 0.8:
            ev3.screen.draw_text(1,50, "Shot 3 Mana Orbs")
            ev3.speaker.say("Ow")
            time.sleep(1)
            ev3.screen.clear()
            ev3.screen.draw_text(1,50, "Dealt" + str((PlayerDMG * 3)) + " DMG")
            EnemyHP = EnemyHP - PlayerDMG * 3
            PlayerMANA = PlayerMANA - 30

    if Class == "Priest":
        ev3.screen.draw_text(1,50, "Used Sin Punish")
        ev3.speaker.say("Ow")
        time.sleep(1)
        ev3.screen.clear()
        ev3.screen.draw_text(1,50, "Dealt" + str((PlayerDMG + 2*(EnemyHP/15))) + " DMG")
        EnemyHP = EnemyHP - PlayerDMG + 2*(EnemyHP/15)
        PlayerMANA = PlayerMANA - 25

    return PlayerHP, EnemyHP, PlayerMANA, Stun
```

Obrázek 14 - Funkce postav 1 konec

```
def ability2(PlayerHP, PlayerDMG, PlayerFullHP, EnemyHP, EnemyFull, PlayerMANA, Stun):

    if Class == "Fighter":
        if EnemyHP <= EnemyFull * 0.25:
            ev3.screen.draw_text(1,50, "Executed the enemy")
            EnemyHP = 0
        else:
            EnemyHP = EnemyHP - (PlayerDMG + 15)
            ev3.screen.draw_text(1,50, "Used Axe Slash")
            ev3.speaker.say("OOF")
            time.sleep(1)
            ev3.screen.clear()
            ev3.screen.draw_text(1,50, "Dealt" + str((PlayerDMG + 15)) + " DMG")
            Stun = True
            PlayerMANA = PlayerMANA - 25

    if Class == "Mage":
        if PlayerHP > 10:
            ev3.screen.draw_text(1,50, "Used Light Spear")
            ev3.speaker.say("OOF")
            time.sleep(1)
            ev3.screen.clear()
            ev3.screen.draw_text(1,50, "Dealt" + str((PlayerDMG + 2*(PlayerHP/5))) + " DMG")
            ev3.screen.draw_text(1,65, "And lost 10 HP")
            EnemyHP = EnemyHP - (PlayerDMG + 2*(PlayerHP/5))
            PlayerHP = PlayerHP - 10
            Stun = True
            PlayerMANA = PlayerMANA - 15
        else:
            ev3.screen.draw_text(1,50, "Used Light Spear")
            ev3.speaker.say("OOF")
            time.sleep(1)
            ev3.screen.clear()
            ev3.screen.draw_text(1,50, "Dealt" + str(PlayerDMG) + " DMG")
            EnemyHP = EnemyHP - PlayerDMG
```

Obrázek 15 - Funkce postav 2

```

if Class == "Priest":
    ev3.screen.draw_text(1,50,"Used Blessing")
    ev3.speaker.say("Scum")
    time.sleep(1)
    ev3.screen.clear()
    ev3.screen.draw_text(1,65,"Healed for" + str(20) + "HP")
    PlayerHP = PlayerHP + 20
    Stun = True
    PlayerMANA = Playermana - 15

return PlayerHP, EnemyHP, PlayerMANA, Stun

```

Obrázek 16 - Funkce postav 2 konec

V této části kódů jsou napsané funkce, které slouží jako schopnosti pro využití uživatelem. Jsou dvě funkce, „ability1“ a „ability2“. Tyto dvě funkce jsou dvě schopnosti, které uživatel bude mít k dispozici. Uživatel samozřejmě bude mít třetí útok, ale ten je hodně jednoduchý a využívá jenom proměnnou „PlayerDMG“, takže není potřeba dělat třetí funkci. Tyto dvě funkce nejprve si zjistí pomocí funkce „if“ za jakou postavu hraje uživatel. Dále vykonají útok určený pro postavu, vypíší text pro uživatele na displej robota a robot zahraje zvuk. Některé „if“ funkce mají i „else“ funkci, protože některé postavy mají schopnost s více výsledky podle stavu hráče nebo robota.

```

def numgenerator():
    number = random.randint(1,3)
    return number

#ENEMY ABILITY1
def enemyability1(PlayerHP, EnemyDMG, Difficulty):
    multiplier = 0
    if Difficulty == "Easy":
        multiplier = 1
    if Difficulty == "Medium":
        multiplier = 2
    if Difficulty == "Hard":
        multiplier = 3

    PlayerHP = PlayerHP - EnemyDMG * (1+multiplier/10)
    ev3.screen.draw_text(5,50, "ENEMY USED MULTISLASH")
    ev3.speaker.say("MULTI SLASH")

    motorsword.run_angle(175, 45*multiplier)

    motorsword.run_angle(175, -45*multiplier)

    motorsword.run_angle(175, 45*multiplier)

    motorsword.run_angle(175, -45*multiplier)

    ev3.screen.clear()
    ev3.screen.draw_text(10,50, "DEALT " + str((EnemyDMG * (1+multiplier/10))) + " DMG")

    return PlayerHP

```

Obrázek 17 - Funkce robota

```

#ENEMY ABILITY2
def enemyability2(PlayerHP, Stun, EnemyDMG, Difficulty):
    if Difficulty == "Easy":
        PlayerHP = PlayerHP - (EnemyDMG - 10)*2

    if Difficulty == "Medium":
        PlayerHP = PlayerHP - EnemyDMG * 2

    if Difficulty == "Hard":
        PlayerHP = PlayerHP - EnemyDMG * 2
        Stun = True

    ev3.screen.draw_text(1,50, "ENEMY USED SHIELD BASH")
    ev3.speaker.say("SHIELD BASH")
    motorshield.run_angle(175, -45)

    motorshield.run_angle(175, 45)

    ev3.screen.clear()

    if Difficulty == "Easy":
        ev3.screen.draw_text(10,50, "DEALT " + str(((EnemyDMG - 10)*2)) + " DMG")

    if Difficulty == "Medium":
        ev3.screen.draw_text(10,50, "DEALT " + str((EnemyDMG * 2)) + " DMG")

    if Difficulty == "Hard":
        ev3.screen.draw_text(10,50, "DEALT " + str((EnemyDMG * 2)) + " DMG")
        Stun = True

    return PlayerHP

```

Obrázek 18 - Funkce robota 2

```

#ENEMY ULTIMATE ABILITY
def enemyultimate(PlayerHP, Difficulty):
    DMG = 0
    if Difficulty == "Easy":
        DMG = 40
    if Difficulty == "Medium":
        DMG = 50
    if Difficulty == "Hard":
        DMG = 60

    PlayerHP = PlayerHP - DMG

    ev3.screen.draw_text(10,50, "ENEMY USED ULTIMATE")
    ev3.speaker.say("Bow before my ultimate attack")

    motorshield.run_angle(175, -60)

    motorsword.run_angle(175, 90)

    motorsword.run_angle(175, -90)

    motorsword.run_angle(175, 90)

    motorsword.run_angle(175, -90)

    motorshield.run_angle(175, 60)

    ev3.screen.clear()
    ev3.screen.draw_text(10,50, "DEALT " + str(DMG) + " DMG")

    return PlayerHP

```

Obrázek 19 - Funkce robota 3

V další sekci jsou deklarované dvě důležité věci pro robota, proti kterému uživatel bojuje. První je funkce „numgenerator“, která slouží pro generování náhodného čísla, a podle čísla si robot vybere, jak bude útočit. Robot má celkem čtyři útoky, ale jeden útok je jednoduchý, takže zase se nepotřebuje funkce navíc.

Funkce „enemyability1“ a „enemyability2“ fungují v principu stejně. Nejprve se zjistí pomocí funkce „if“, jaká je složitost, protože uživatel si bude vybírat složitost souboje proti robotovi. Podle složitosti, hodnoty útoků se budou měnit. Dále se provede útok, na displej se vypíší informace pro uživatele, robot vydá zvuk a také vykoná pohyb motorem (záleží na útoku). Robot má také funkci „enemyultimate“, což je jeho ultimátní útok, který funguje stejně jako minulé útoky, ale tentokrát vykoná pohyb oběma motory.


```

def detect_color():
    color = color_sensor.color()
    if color == color.RED:
        return "RED"
    elif color == color.GREEN:
        return "GREEN"
    elif color == color.BLUE:
        return "BLUE"

ev3.speaker.beep()

ev3.speaker.say("Welcome, my name is Battl-o-tron and I will fight you")
time.sleep(1)
ev3.screen.draw_text(10,20, "Select a difficulty")
ev3.screen.draw_text(10,35, "R - Easy")
ev3.screen.draw_text(10,50, "G - Medium")
ev3.screen.draw_text(10,65, "B - Hard")
ev3.speaker.say("Please select a difficulty")

while True:
    if button.pressed():
        break

ev3.screen.clear()

```

Obrázek 20 - Začátek hlavního programu

```

def detect_color():
    if detect_color() == "RED":
        Difficulty = "Easy"
        EnemyHP = 140
        EnemyDMG = 15
        EnemyHPFull = 140
        ev3.screen.draw_text(5,20,"Difficulty is Easy")
        ev3.screen.draw_text(5,35, "Stuns: Disabled")
        ev3.speaker.say("You have selected Easy")

    elif detect_color() == "GREEN":
        Difficulty = "Medium"
        EnemyHP = 160
        EnemyDMG = 20
        EnemyHPFull = 160
        ev3.screen.draw_text(5,20,"Difficulty is Medium ")
        ev3.screen.draw_text(5,35, "Stuns: Disabled")
        ev3.speaker.say("You have selected Medium")

    elif detect_color() == "BLUE":
        Difficulty = "Hard"
        EnemyHP = 190
        EnemyDMG = 25
        EnemyHPFull = 190
        ev3.screen.draw_text(5,20,"Difficulty is Hard ")
        ev3.screen.draw_text(5,35, "Stuns: Enabled")
        ev3.speaker.say("You have selected Hard")

    time.sleep(2)
    ev3.screen.clear()

```

Obrázek 21 - Výběr složitosti

Na této sekci je deklarovaná hodně důležitá funkce a zde začíná hra pro hráče. Funkce „detect_color“ slouží pro uživatele pro výběr akce. Ta vrátí hodnotu typu „string“ podle toho, jaká je detekovaná barva senzorem (Red, Green, Blue). Poněvadž senzor detekuje barvy neustále v reálném čase, většinou v kódu se bude nacházet nekonečný „while“ cyklus, který obsahuje „if“ funkci, která pustí program dál při zmáčknutí tlačítka. Po tomto „while“ cyklu se bude nacházet samotná funkce „detect_color“, která senzorem nadetekuje barvu a vrátí ji jako „string“ hodnotu.

Toto je vidět dál na těchto obrázcích. Uživatele nejprve robot uvítá a zeptá se o výběr složitosti (jsou tři složitosti). Uživatel stiskne tlačítko, nadetekuje se barva a podle toho program vybere

složitost. Podle složitosti se mění životy a síla robota, také se povoluje či nepovoluje „Stun“ mechanika. Na displej se vypíše vybraná složitost a robot řekne vybranou složitost.

```

Proceed = False
while Proceed != True:
    ev3.screen.draw_text(10,20, "Select a class")
    ev3.screen.draw_text(10,35, "R - Fighter")
    ev3.screen.draw_text(10,50, "G - Mage")
    ev3.screen.draw_text(10,65, "B - Priest")
    ev3.speaker.say("Please select your class")

    while True:
        if button.pressed():
            break

    ev3.screen.clear()

def detect_color():
if detect_color() == "RED":
    Class = "Fighter"
    PlayerHP = 150
    PlayerDMG = 20
    PlayerHPFull = 150
    PlayerMANA = 100
    PlayerMANAFull = 100
    Manacost1 = 20
    Manacost2 = 25
    Basic = "Punch"
    Ab1 = "Brute Strength"
    Ab2 = "Axe Slash"
    ev3.screen.draw_text(20,5,"Fighter")
    ev3.speaker.say("You are a Fighter")

    ev3.screen.draw_text(1,20,"Brute Strength") #Deal DMG based on your HP
    ev3.screen.draw_text(1,35,"Axe Slash") #15 more DMG but execute if enemy is %25 below
    ev3.screen.draw_text(1,50,"Punch") #Deal 20 DMG

```

Obrázek 22 - Výběr postavy

```

elif detect_color() == "GREEN":
    Class = "Mage"
    PlayerHP = 115
    PlayerDMG = 15
    PlayerHPFull = 100
    PlayerMANA = 120
    PlayerMANAFull = 120
    Manacost1 = 30
    Manacost2 = 15
    Basic = "Mana Beam"
    Ab1 = "Mana Orbs"
    Ab2 = "Light Spear"
    ev3.screen.draw_text(20,5,"Mage")
    ev3.speaker.say("You are a Mage")

    ev3.screen.draw_text(1,20,"Mana Orbs") #DMG based on enemy HP
    ev3.screen.draw_text(1,35,"Light Spear") #DMG based on your HP but lose HP
    ev3.screen.draw_text(1,50,"Mana Beam") #Deal 15 DMG

```

Obrázek 23 - Výběr postavy 2

```

elif detect_color() == "BLUE":
    Class = "Priest"
    PlayerHP = 120
    PlayerDMG = 10
    PlayerHPFull = 120
    PlayerMANA = 90
    PlayerMANAFull = 90
    Manacost1 = 25
    Manacost2 = 15
    Basic = "Curse"
    Ab1 = "Sin Punish"
    Ab2 = "Blessing"
    ev3.screen.draw_text(30,10,"Priest")
    ev3.speaker.say("You are a Priest")

    ev3.screen.draw_text(1,20,"Sin Punish") #Additional DMG based on enemy HP
    ev3.screen.draw_text(1,40,"Blessing") #Heal yourself
    ev3.screen.draw_text(1,60,"Curse") #Deal 10 DMG

    ev3.screen.draw_text(1,80,"PRESS to continue")
    ev3.screen.draw_text(1,95,"HOLD to return")
    while True:
        if button.pressed():
            Proceed = True
            time.sleep(2)
        if button.pressed():
            Proceed = False
            break

    ev3.screen.clear()

```

Obrázek 24 - Výběr postavy 3

V této sekci programu si uživatel vybírá postavu. Vypíše se text a ukáže jakou barvou si uživatel by měl vybrat určitou postavu. Pro výběr jsou 3 postavy, Fighter, Mage a Priest. Zde je zase vidět zase využití tlačítka a potom detekce barev. Podle detekované barvy se nastaví postava. Pro určení postavy program si uloží její jméno do hodnoty typu „string“, která se jmenuje „Class“. Také se vypíší názvy schopností postavy.

V této části uživatel má také možnost se vrátit na začátek tohoto výběru. Po výběru postavy uživatel musí potvrdit výběr nebo se vrátit. Potvrzení je jedno zmáčknutí tlačítka, zatímco pro vrácení uživatel musí tlačítko podržet. Toto je uděláno pomocí „while“ cyklu s podmínkou, která se mění podle zmáčknutí tlačítka.

```

ev3.screen.draw_text(10, 50, "BATTLE BEGIN")
ev3.speaker.say("PROVE YOURSELF AND FIGHT ME")
ev3.screen.clear()
time.sleep(1)
ev3.speaker.beep()

while PlayerHP > 0 and EnemyHP > 0:

    Proceed = False

    if Stun == True and StunUsedEnemy == False and Difficulty == "Hard":
        Proceed = True
        StunUsedEnemy = True
        ev3.screen.draw_text(20,20, "YOU ARE STUNNED")
        time.sleep(2)
        ev3.screen.clear()
        Stun = False
    else:
        StunUsedEnemy = False
        ev3.screen.draw_text(20,20, "YOUR TURN")
        time.sleep(1)

    while Proceed != True:
        ev3.screen.clear()
        ev3.screen.draw_text(1,5, "HP: " + str(PlayerHP))
        ev3.screen.draw_text(1,20, "MANA: " + str(PlayerMANA))
        ev3.screen.draw_text(1,35, "Robot HP: " + str(EnemyHP))
        ev3.screen.draw_text(1,50, "R: " + str(Basic))
        ev3.screen.draw_text(1,65, "G: " + str(Ab1) + " | M: " + str(Manacost1))
        ev3.screen.draw_text(1,80, "B: " + str(Ab2) + " | M: " + str(Manacost2))

```

Obrázek 25 - Začátek souboje

```

while True:
    if button.pressed():
        break

    ev3.screen.clear()
    detect_color()
    if detect_color() == "RED":
        EnemyHP = EnemyHP - PlayerDMG
        ev3.screen.draw_text(1,50, "You used " + str(Basic))
        time.sleep(1)
        ev3.screen.draw_text(1,50, "Dealt " + str(PlayerDMG))
        time.sleep(2)
        Proceed = True

    elif detect_color() == "GREEN":
        if PlayerMANA >= Manacost1:
            PlayerHP, EnemyHP, PlayerMANA, Stun = ability1(PlayerHP, PlayerDMG, PlayerHPFull, EnemyHP, EnemyHPFull, PlayerMANA, Stun)
            time.sleep(2)
            Proceed = True
        else:
            ev3.screen.draw_text(1,50, "NOT ENOUGH MANA")
            time.sleep(1)

    elif detect_color() == "BLUE":
        if PlayerMANA >= Manacost2:
            PlayerHP, EnemyHP, PlayerMANA, Stun = ability2(PlayerHP, PlayerDMG, PlayerHPFull, EnemyHP, EnemyHPFull, PlayerMANA, Stun)
            time.sleep(2)
            Proceed = True
        else:
            ev3.screen.draw_text(1,50, "NOT ENOUGH MANA")
            time.sleep(1)

    ev3.screen.clear()

```

Obrázek 26 - Výběr hráče

```

if EnemyHP <= 0:
    break
else:
    EnemyTurn = True

    if Stun == True and StunUsedPlayer == False and Difficulty == "Hard":
        EnemyTurn = False
        StunUsedPlayer = True
        ev3.screen.draw_text(20,20, "ENEMY STUNNED")
        time.sleep(2)
        ev3.screen.clear()
        Stun = False
    else:
        StunUsedPlayer = False
        ev3.screen.draw_text(20,20, "ENEMY TURN")
        time.sleep(2)
        ev3.screen.clear()

```

Obrázek 27 - Ověření tahu robota

```

while EnemyTurn == True:                                     #ENEMYTURN
    ultimate = random.randint(1,10)
    if ultimate == 5:
        PlayerHP = enemyultimate(PlayerHP, Difficulty)
        EnemyTurn = False
    else:
        numgenerator()
        if numgenerator == 1:
            PlayerHP = PlayerHP - EnemyDMG
            ev3.screen.draw_text(10,50, "ENEMY USED BASIC")
            ev3.speaker.say("EN GUARDE")
            motorsword.run_angle(175, 45)
            motorsword.run_angle(175, -45)
            ev3.screen.clear()
            ev3.screen.draw_text(10,50, "DEALT " + str(EnemyDMG) + " DMG")
            time.sleep(2)
            EnemyTurn = False
        elif numgenerator == 2:
            PlayerHP = enemyability1(PlayerHP, EnemyDMG, Difficulty)
            time.sleep(2)
            EnemyTurn = False
        elif numgenerator == 3:
            PlayerHP = enemyability2(PlayerHP, EnemyDMG, Difficulty)
            time.sleep(2)
            EnemyTurn = False
    ev3.screen.clear()

if PlayerMANA != PlayerMANAFull:
    if PlayerMANA >= (PlayerMANAFull-10):
        PlayerMANA = PlayerMANAFull
    else:
        PlayerMANA = PlayerMANA + 10

if PlayerHP <= 0:
    break

```

Obrázek 28 - Tah robota

Tato část je nejhlavnější část programu, zde uživatele vyzve robot na souboj. Souboj je ve „while“ cyklu s podmínkou, že uživatel a robot jsou naživu. Před tahem uživatele se nachází kontrola „Stun“ mechaniky, ta funguje pouze při složitosti „Hard“ a pokud robot nepoužil „Stun“ v minulém tahu, aby nedošlo k více omráčením za sebou. Tato mechanika se uplatňuje i před tahem robota a funguje stejně. Program zde koná pokyny podle detekovaných barev, tedy akce uživatele. U tahu robota se nejprve vybere číslo od 1 do 10, pokud číslo je 5, tak robot použije ultimátní útok. Pokud ne, tak se vybere číslo od 1 do 3 pomocí „numgenerator“ a robot udělá jeden ze standartních útoků. Po tahu robota se uživateli regeneruje mana a program se vrací na začátek cyklu.

```
if PlayerHP > 0 and EnemyHP <= 0:
    ev3.screen.clear()
    ev3.screen.draw_text(20, 50, "YOU WIN")
    ev3.speaker.say("YOU WIN")
    ev3.speaker.say("FAREWELL, PLAYER")

elif PlayerHP <= 0 and EnemyHP > 0:
    ev3.screen.clear()
    ev3.screen.draw_text(20, 50, "YOU LOSE")
    ev3.speaker.say("YOU LOSE, HAAAAA LOSER, LOSER, LOSER")

exit()
```

Obrázek 29 - Konec programu

Pokud uživatel nebo robot zemře, tak program vyskočí z cyklu a pokračuje sem, kde zjistí, kdo vyhrál. Pokud vyhrál uživatel, tak robot řekne, že uživatel vyhrál a rozloučí se s ním. Pokud vyhrál robot, tak se uživateli vysměje a řekne, že prohrál. Potom se program ukončí. Z původního návrhu se nestihla doprogramovat a vytestovat finální animace robota.

4 Manuál

Robot se nejprve spustí a uvítá uživatele. Hned na začátku se zeptá pro výběr složitosti od uživatele, toto mění, jak silný robot bude a jak těžko ho bude porazit. Pro výběr možností a dalších akcí při souboji s robotem se využívá tlačítkový senzor a senzor barev. Většinou se využívají tak, že nejprve uživatel musí přiložit barvu korespondující k výběru, který je vypsán v daném momentě na displeji robota.

Existují tři složitosti, Easy, Medium a Hard. Potom se zmáčkne tlačítko a výběr se potvrdí. Toto funguje stejně v dalším kroku, kde robot se zeptá uživatele, za jakou postavu chce uživatel hrát. Na výběr jsou tři postavy, Fighter, Mage a Priest. Po výběru postavy se ukáží její názvy schopností a uživatel pro pokračování musí jenom zmáchnout tlačítko. Ale uživatel si může vybrat jinou postavu, pokud tlačítko podrží, dokud ho program nevrátí zpět k výběru postav.

Složitost mění robotovi životy a sílu, taky trochu i funkcionalitu některých jeho schopností. Při složitosti „Easy“ robot má 140 životů a 15 síly. Při složitosti „Medium“ robot má 160 životů a 20 síly. Při složitosti „Hard“ robot má 190 životů a 25 síly. Při složitosti „Hard“ je taky zapnutá mechanika „Stun“ pro omráčení, tedy přeskok tahu robota nebo uživatele.

Každá postava má jiné životy, sílu a počet many. Zde jsou jejich parametry a co dělají jejich jednotlivé schopnosti. Každá postava má 3 útoky, jeden z nich je obecný, který jenom využívá hodnotu síly a nepotřebuje manu.

Fighter obsahuje 150 životů, 20 síly a 100 many. Jeho obecný útok se jmenuje „Punch“, který ubere robotovi 20 životů. Jeho první schopnost „Brute Strength“, ta ubírá robotovi životy podle síly a navíc 15 % z původních životů uživatele a stojí 20 many. Druhá schopnost „Axe Slash“ ubírá životy podle síly a navíc 15 životů, ale pokud robot má méně jak čtvrtinu původních životů, tak ho tato schopnost hned zabije a proto stojí 25 many a omračuje robota při složitosti „Hard“.

Mage obsahuje 115 životů, 15 síly a 120 many. Obecný útok se jmenuje „Mana Beam“ a ubírá 15 životů robotovi. Jeho první schopnost „Mana Orbs“ ubírá robotovi životy podle toho kolik životů mu zbývá. Čím víc zbývajících životů, tím větší poškození. Pokud robot má pod 20 % životů, tak se ubere jenom 15. Stojí 30 many. Druhá schopnost „Light Spear“ ubere životy podle síly, a navíc podle zbývajících životů uživatele, čím víc životů tím větší poškození. Ale tato schopnost také ubere uživateli 10 životů. Tato schopnost také omračuje při složitosti „Hard“ a stojí 15 many.

Priest obsahuje 120 životů, 10 síly a 90 many. Obecný útok se jmenuje „Curse“ a ubírá 10 životů robotovi. První schopnost „Sin Punish“ ubírá životy podle síly, a navíc podle zbývajících životů robota, čím víc životů tím větší poškození a stojí 20 many. Druhá schopnost „Blessing“ uživatele vyléčí a přidá 20 životů. Stojí také 20 many a omračuje robota při složitosti „Hard“.

Když začne souboj, tak uživatel hraje první a vybírá útoky podle tří barev (Red, Green, Blue) stejně jako u výběru složitostí a postav. Po výběru útoku se útok provede, pokud nestačí mana, uživatel musí vybrat jiný útok. Dále hraje robot, který vybírá útoky náhodně a má 10 % šanci udělat ultimátní útok. Jeho útoky mají větší poškození při větších složitostech a také obsahuje schopnost, která omračuje hráče. Robot podle schopnosti vydá zvuk a určitou animaci. „Stun“ mechanika omráčí jenom na jeden tah, a použití schopnosti s omráčením dvakrát za sebou podruhé už neomráčí. Boj skončí, když uživatel nebo robot zemře. Takhle skončí celý program.

Pokud robot nebo program nějak nefunguje, nejlépe je ho restartovat a ujistit si, že všechny součástky jsou správně zapojeny do svých portů. Začáteční pozice motoru štítu musí být tak, aby štít omylem nenarazil do samotného motoru, poněvadž se pohybuje o 60 stupňů.

5 Závěr

Očekávání se víceméně určitě vyplnilo až na pár věcí, se kterými se na začátku počítalo ale poté zjistilo, že do robota buď nepůjdou nebo je moc těžké je tam dát. Tato situace se odehrála například se sound efekty, které měl robot mít. V této sekci se by mohlo říct, že nejdelší a nejtěžší část realizování tohoto projektu byl samotný program, poněvadž je to nejhlavnější a největší záležitost celého projektu. Nejsnadnější byl definitivně úvod, jelikož do projektu se šlo s jasným nápadem a plánem. Podle dohodnutí by se moc úprav neudělalo, protože finální myšlenka je, že projekt je docela povedený, možná by se mělo dodat více hýbajících se částí na robota, aby byl zážitek ze hry o to zajímavější. Dodatků se k tomuto projektu dá udělat opravdu mnoho. V této verzi lze s robotem hrát pouze jednu bitvu, ale v pozdějších iteracích by robot mohl vzít roli vypravěče a hrát s vámi celou RPG hru což podle obecné logiky by bylo mnohem zajímavější pro zážitek hráče, který nejspíš má větší očekávání od novějších technologií.

6 Seznam použitých informačních zdrojů

Lego Mindstorms Education EV3 MicroPython [online]. [cit. 2024-5-14]

Dostupné v: <https://pybricks.com/ev3-micropython/>

Lego Mindstorms Education, Python for EV3 [online]. [cit. 2024-5-14]

Dostupné v: <https://education.lego.com/en-us/product-resources/mindstorms-ev3/teacher-resources/python-for-ev3/>

Pomoc od umělé inteligence s programováním, ChatGPT [online]. [cit. 2024-5-14]

Dostupné v: <https://chatgpt.com>

7 Seznam příloh

Visual Studio Code. Dostupné v: <https://code.visualstudio.com>

EV3 MicroPython rozšíření. Dostupné v prohlížeči „Extensions“ ve Visual Studio Code

8 Seznam obrázků

Obrázek 1 - Nakreslený návrh konstrukce	3
Obrázek 2 - Celý vývojový diagram	4
Obrázek 3 - Začátek diagramu.....	5
Obrázek 4 - Tah hráče v diagramu	6
Obrázek 5 - Tah robota v diagramu.....	7
Obrázek 6 - Konec diagramu.....	8
Obrázek 7 - Nadefinované funkce v diagramu.....	8
Obrázek 8 - Kabely robota seshora.....	9
Obrázek 9 - Kabely robota zezadu	10
Obrázek 10 - Finální konstrukce	10
Obrázek 11 - Knihovny programu.....	11
Obrázek 12 - Nadefinované hodnoty	11
Obrázek 13 - Funkce postav 1	12
Obrázek 14 - Funkce postav 1 konec.....	12
Obrázek 15 - Funkce postav 2	12
Obrázek 16 - Funkce postav 2 konec.....	13
Obrázek 17 - Funkce robota	14
Obrázek 18 - Funkce robota 2	14
Obrázek 19 - Funkce robota 3	15
Obrázek 20 - Začátek hlavního programu	16
Obrázek 21 - Výběr složitostí.....	16
Obrázek 22 - Výběr postavy	17
Obrázek 23 - Výběr postavy 2.....	17
Obrázek 24 - Výběr postavy 3	17
Obrázek 25 - Začátek souboje	19
Obrázek 26 - Výběr hráče.....	19
Obrázek 27 - Ověření tahu robota	19
Obrázek 28 - Tah robota.....	20

Obrázek 29 - Konec programu	21
-----------------------------------	----